Experiment Title:

HTTP Client–Server Configuration in Java

Introduction

Client–Server architecture is a fundamental concept in Computer Networks where a **client sends requests** and a **server responds** to those requests over a network.

The **HTTP (HyperText Transfer Protocol)** is widely used for communication between web clients (like browsers) and servers. It follows a **request–response model**, where the client sends an HTTP request and the server returns an HTTP response.

In this experiment, a simple HTTP server is created using Java's built-in HttpServer class, and a client is implemented using HttpURLConnection to establish communication between them.

Procedure (Code)

Step 1: Create HTTP Server

```
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;
import java.io.*;
import java.net.InetSocketAddress;

public class SimpleHttpServer {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new
InetSocketAddress(5000), 0);

        server.createContext("/", new MyHandler());
        server.setExecutor(null);
        server.start();
```

```
        System.out.println("Server started at
http://localhost:5000");
    }

    static class MyHandler implements HttpHandler {
        public void handle(HttpExchange exchange) throws
IOException {

            String response = "HTTP Server running successfully
on port 5000.\nClient-Server communication established.";

            exchange.getResponseHeaders().set("Content-Type",
"text/plain");

            byte[] responseBytes = response.getBytes();

            exchange.sendResponseHeaders(200,
responseBytes.length);

            OutputStream os = exchange.getResponseBody();
            os.write(responseBytes);
            os.close();
        }
    }
}
```

Step 2: Create HTTP Client

```
import java.io.*;
import java.net.*;

public class SimpleHttpClient {
```

```
    public static void main(String[] args) throws Exception {

        URL url = new URL("http://localhost:5000/");
        HttpURLConnection conn = (HttpURLConnection)
url.openConnection();

        conn.setRequestMethod("GET");

        int status = conn.getResponseCode();
        System.out.println("Response Code: " + status);

        BufferedReader in = new BufferedReader(
                new InputStreamReader(conn.getInputStream())
        );

        String inputLine;
        System.out.println("Response from server:");

        while ((inputLine = in.readLine()) != null) {
            System.out.println(inputLine);
        }

        in.close();
        conn.disconnect();
    }
}
```

Step 3: Execution Steps

1. Compile both files
2. Run the server
3. Run the client
4. Observe the communication output

Output

Console Output

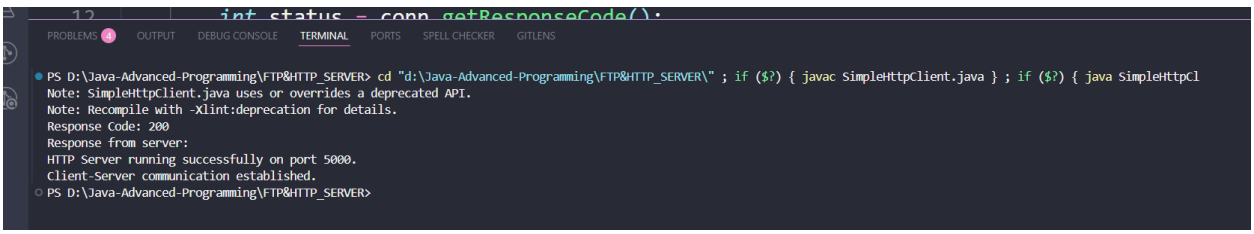
Server started at http://localhost:5000

Response Code: 200
Response from server:
HTTP Server running successfully on port 5000.
Client-Server communication established.

Screenshot

- ✓ Server running terminal

- ✓ Client output terminal



Conclusion

In this experiment, a simple HTTP client–server communication system was successfully implemented using Java. The server handled incoming HTTP requests and responded with appropriate messages, while the client successfully sent requests and received responses.

This experiment helped in understanding the working of the HTTP protocol, client–server architecture, and basic network communication in Java.